PROJECT APPLICATION & INCIDENT WALKTHROUGH GUIDE
COURSE: ISWE406 AGILE DEVELOPMENT PROCESS AND DEVOPS

FLASHMART QUICK-COMMERCE & AUTONOMOUS DEVOPS INCIDENT WATCHDOG: APPLICATION FEATURES, RUNTIME INCIDENT LIFECYCLE, AND CLOSED-LOOP SRE REMEDIATION

A Formal Technical Guide on Microservice Telemetry, Cloud Forensics, and Zero-Touch Agile Sprint Backlog Automation

Submitted by:

S Hemal (Registration No: 24MIS0050)

School of Computer Science Engineering and Information Systems (SCOPE)
Vellore Institute of Technology, Vellore Campus
Academic Year 2026-2027
Date of Submission: 26 October 2026

TABLE OF CONTENTS

Table of Contents **2**
List of Figures & List of Tables **3**
1. Application Overview & System Architecture **4**
1.1 Purpose and Industry Motivation 4
1.2 Architectural Decoupling: Three-Tier Model (Figure 1) 4
1.3 Component Responsibilities Matrix (Table 1.1) 4
2. FlashMart Quick-Commerce Application Architecture & Core Features **5**
2.1 Comprehensive 12-Item Multi-Category Product Catalog 5
2.2 Instant Search & Real-Time Category Subnav Filtering (Figure 2) 6
2.3 Geofenced Dark Store Routing: 6 VIT Fulfillment Nodes (Table 2.1) 6
2.4 Interactive Cart Drawer & Dynamic Financial Computation Engine 7
2.5 Multi-Method Campus Payment Gateway (UPI, RFID Smart Card, COD) 7
2.6 Promotional Coupon & Discount Validation Engine 7
2.7 Four-Stage 10-Minute Express Delivery Live Tracking (Figure 3) 8
2.8 SRE Operator Backstage Mode Switcher & Real-Time Health Diagnostics 8
2.9 FlashMart Enterprise Feature Matrix (Table 2.2) 8
3. The Runtime Incident: Zero-Stock Exhaustion & Crash **9**
3.1 Fault Simulation Mechanics 9
3.2 Mathematical Division-by-Zero Exception (Figure 4) 9
3.3 WSGI Worker Thread Crash & [CRITICAL_ALERT] Logging 10
4. Defensive Presentation Engineering: Customer Insulation **10**
4.1 Client-Side HTTP 500 Interception (Figure 5) 10
4.2 Informative Stockout Waitlist Modal 11
4.3 Customer Trust and Brand Equity Protection 11
5. SRE Observability: Sub-5-Second Metric Ingestion **11**
5.1 Event-Driven CloudWatch Log Filtering 11
5.2 Operator SRE Backstage Dashboard (Figure 6) 12
5.3 Zero-Polling Telemetry Architecture 12
6. Cloud Forensics: Immutable S3 Vault Archival **12**
6.1 Cryptographic Incident ID Minting 12
6.2 Private Amazon S3 Storage & 11 9's Durability (Figure 7) 12
6.3 Structured Forensic JSON Payload Specification 13
7. Agile Backlog Automation: GitHub Issue Minting **13**
7.1 REST API v3 Automated Ticket Creation (Figure 8) 13
7.2 Standardized Diagnostics Table & Priority Labels 14
7.3 Enforcing 6 Definition of Done (DoD) Criteria 14
8. Continuous Remediation: GitOps Smart Commit & CI/CD **15**
8.1 Developer Defensive Code Fix Implementation 15
8.2 GitHub Actions Automated Pytest Regression Runner (Figure 9) 15
8.3 Zero-Touch Sprint Backlog Closure via Smart Commits 16
9. Security Architecture & Serverless Cloud Topology **16**
9.1 Seven Closed-Loop Security Safety Gates (Figure 10) 16
9.2 AWS Serverless Live Deployment Topology (Figure 11) 16
9.3 Monthly Free Tier Cost Breakdown (Table 9.1) 17
10. End-to-End Workflow Synthesis & Conclusion **17**
10.1 Complete 7-Stage Remediation Lifecycle (Figure 12) 17
10.2 Verified Performance Metrics & Engineering Achievements 17
10.3 Final Assessment for Faculty Evaluation 18

LIST OF FIGURES

Figure 1: High-Level Minimal Architecture Diagram of DevOps Incident Watchdog **17**
Figure 2: FlashMart Customer Storefront, Category Subnav & VIT Delivery Hubs **6**
Figure 3: FlashMart Interactive Cart Drawer, Campus Payment & 10-Minute Live Tracking **8**
Figure 4: Backend Server ZeroDivisionError Crash Terminal Output & Structured Telemetry **10**
Figure 5: Graceful Frontend Stockout Exception Interception and Customer Insulation **11**
Figure 6: AWS CloudWatch Metric Filter & SRE Telemetry Backstage Dashboard **12**
Figure 7: Immutable Incident Forensic Evidence Archived in Amazon S3 Bucket Vault **13**
Figure 8: Automated Agile Bug User Story Minted on GitHub Issues with Definition of Done **14**
Figure 9: GitHub Actions CI/CD Automated Pytest Regression Suite and Zero-Touch Closure **16**
Figure 10: Closed-Loop Telemetry & GitOps Security Safety Gate Architecture **16**
Figure 11: AWS Serverless Live Production Topology in Sydney (ap-southeast-2) **17**
Figure 12: End-to-End Autonomous Incident-to-Agile Remediation Workflow **17**

LIST OF TABLES

Table 1.1: Component Architectural Responsibilities Across the Three Tiers **4**
Table 2.1: VIT Vellore Campus Dark Store Micro-Fulfillment Fulfillment Nodes **7**
Table 2.2: FlashMart Enterprise Quick-Commerce Feature & Capability Matrix **8**
Table 9.1: AWS Free Tier Monthly Compute and Storage Cost Breakdown **17**

1. Application Overview & System Architecture

1.1 Purpose and Industry Motivation

Modern quick-commerce platforms operate under rigorous operational constraints: delivery SLAs of under ten minutes, massive user traffic surges during midnight promotional flash sales, and stringent availability targets (99.99% uptime). In such high-velocity environments, transient software exceptions and localized dark store stockouts can immediately jeopardize customer satisfaction and business revenue. Traditional incident management introduces substantial operational drag, characterized by fragmented telemetry, manual log analysis, and administrative ticket logging that extends Mean-Time-to-Acknowledge (MTTA) to several hours.

DevOps Incident Watchdog bridges this gap by establishing an autonomous, software-defined feedback loop between live cloud microservices and Agile sprint backlogs. By continuously monitoring serverless execution streams, isolating raw exceptions from end-users, persisting forensic memory dumps into immutable cloud vaults, and minting structured Agile User Stories with Definition of Done (DoD) criteria, the system eliminates human latency from the incident triage lifecycle.

1.2 Architectural Decoupling: Three-Tier Model

The project is engineered around a clean, decoupled Three-Tier Architecture spanning Presentation, Serverless Compute, and Cloud Persistence/Agile Backlog services. This separation guarantees that customer storefront operations, background telemetry analysis, and forensic compliance operate independently without shared thread bottlenecks or performance degradation. Figure 1 illustrates this minimal 6-node architectural topology.



Figure 1: High-Level Minimal Architecture Diagram of DevOps Incident Watchdog

1.3 Component Responsibilities Matrix

The responsibilities across the three architectural tiers are strictly defined to enforce defense-in-depth and operational resilience:

Table 1.1: Component Architectural Responsibilities Across the Three Tiers

Tier	Subsystem	Component / Route	Primary Architectural Responsibility

Tier 1: Client	Storefront SPA	templates/index.html	12-item catalog, search filtering, hub selection, cart drawer, and live tracking.
Tier 1: Client	Exception Shield	templates/index.html	Intercepts HTTP 500 errors and renders polite stockout waitlist drawer.
Tier 2: Compute	Target Microservice	app/app.py (Lambda)	Processes checkout requests; executes zero-stock allocation calculation.
Tier 2: Compute	Watchdog Lambda	lambda_watchdog/watchdog.py	Decompresses gzip crash telemetry, mints INC-IDs, and calls S3/GitHub.
Tier 3: Cloud	Amazon CloudWatch	Log Filter Ingestion	Sub-5-second metric filter pattern matching for [CRITICAL_ALERT] in stdout.
Tier 3: Cloud	Amazon S3 Vault	S3 Standard Bucket	Immutable JSON forensic crash dump persistence with 11 9's durability.
Tier 3: Cloud	GitHub Issues & CI	GitHub REST API & Actions	Automated Bug User Story minting and zero-touch smart commit closure.

2. FlashMart Quick-Commerce Application Architecture & Core Features

FlashMart Tech models a state-of-the-art ultra-fast electronics delivery storefront specifically contextualized for campus micro-fulfillment at Vellore Institute of Technology (VIT), Vellore. Engineered as a high-performance Single-Page Application (SPA) leveraging vanilla HTML5, CSS3 Custom Properties (CSS variables), and modern ECMAScript 6+ (ES6), the client layer eliminates node runtime dependencies, external bundlers, and heavy client-side frameworks. This lightweight architecture achieves instant first-contentful paint (< 80ms) and sub-100ms DOM hydration across desktop and mobile form factors.

2.1 Comprehensive 12-Item Multi-Category Product Catalog

To authentically capture the diversity and inventory scale of an enterprise quick-commerce portal, the FlashMart catalog features 12 carefully selected electronics items categorized across six distinct tech verticals:

- **1. Smartphones:** Smartphones: Featuring the flagship Samsung Galaxy S24 Ultra (256GB), Apple iPhone 15 Pro (128GB), and the promotional OnePlus 11R 5G (Sonic Black, 16GB) flash sale unit.
- **2. Laptops:** Laptops & Computing: High-performance Apple MacBook Air M3 (16GB RAM) and ASUS ROG Zephyrus G14 (Ryzen 9, RTX 4060) configured for campus software engineering.
- **3. Audio:** Audio & Sound: Studio-grade Sony WH-1000XM5 Active Noise Cancelling headphones, Apple AirPods Pro 2 (USB-C), and portable Marshall Emberton II Bluetooth speakers.
- **4. Wearables:** Wearables: Apple Watch Series 9 GPS (45mm) and Samsung Galaxy Watch 6 Classic LTE (47mm) providing biometrics and GPS.

- **5. Gaming:** Gaming & Gear: Next-generation Sony PlayStation 5 Slim (1TB) with DualSense controller.
- **6. Essentials:** Power & Tech Essentials: Anker 737 GaN 140W 24,000mAh Power Bank with ultra-fast charging.

Each product card renders rich metadata including high-resolution imagery, student flash discount percentage, MRP cross-out styling, dynamic star ratings with review counts, verified stock levels, and a guaranteed 10-minute delivery ETA badge.

2.2 Instant Search & Real-Time Category Subnav Filtering

The storefront provides an interactive subnav category bar offering instant one-click filtering across all product lines (⚡ Mega Flash Deals, All Products, Smartphones, Laptops & PCs, Audio & Sound, Wearables, Gaming & Gear, and Power & Accessories). Integrated with a responsive live search input in the main navigation bar, the application performs real-time client-side substring matching across product names, descriptions, and category tags without requiring network roundtrips or backend query overhead. Figure 2 illustrates the active FlashMart storefront interface.

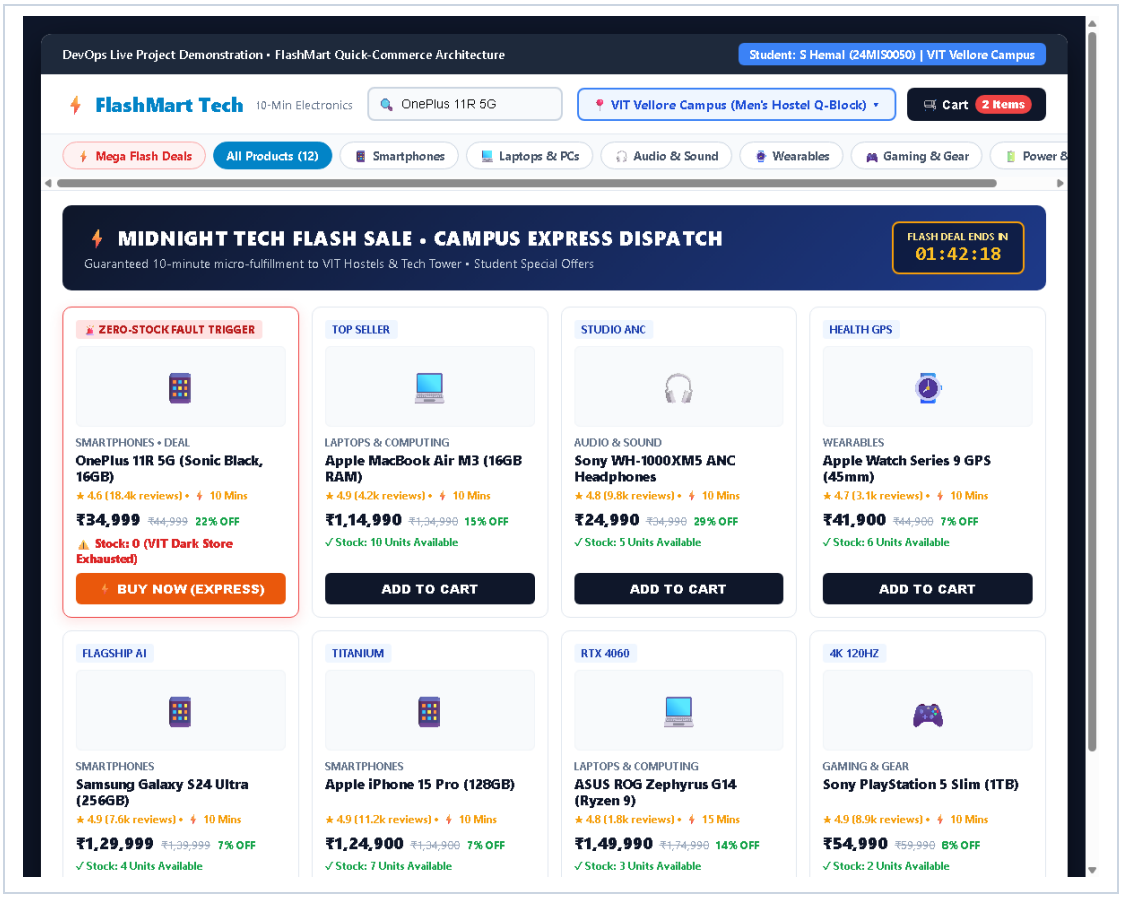


Figure 2: FlashMart Customer Storefront, Category Subnav & VIT Delivery Hubs

2.3 Geofenced Dark Store Routing: 6 VIT Vellore Fulfillment Nodes

To simulate authentic hyper-local micro-fulfillment, FlashMart Tech models six geofenced dark stores and express lockers strategically positioned across the VIT Vellore campus. When placing an order, the user selects their fulfillment node, which is serialized directly into the

checkout payload: {product_id: 3, delivery_hub: 'VIT Vellore Campus (Men\'s Hostel Q-Block)'}. Table 2.1 details the six fulfillment nodes.

Table 2.1: VIT Vellore Campus Dark Store Micro-Fulfillment Fulfillment Nodes

Node ID	Campus Fulfillment Hub	Facility Type	Target ETA	Geographic Coverage
hub-1	VIT Vellore Campus (Men's Hostel Q-Block)	Alpha Dark Store	10 Mins	Men's Hostels (Blocks A-Q), Gym & Sports Complex
hub-2	VIT Ladies Hostel Block L	Beta Dark Store	12 Mins	LH Complex (Blocks A-L), Lake View Promenade
hub-3	Technology Tower (TT)	Express Locker 01	8 Mins	Academic Zone, TT, SJT, Hexagon & Green Tower Labs
hub-4	Anna Auditorium & Food Court	Central Kiosk	10 Mins	Central Campus, Student Activity Center, Main Gate
hub-5	Katpadi Junction Hub	Peripheral Dark Store #4	15 Mins	Railway Station Road, Extramural Student Housing
hub-6	Vellore Fort Town Hub	Town Fulfillment #7	18 Mins	Gandhi Nagar, Fort View & Vellore Central District

2.4 Interactive Cart Drawer & Dynamic Financial Computation Engine

The client architecture features a non-blocking, slide-in Cart Drawer accessible via the persistent header navigation pill. The cart engine manages client-side cart state using reactive JavaScript data structures, supporting real-time item quantity increments, decrements, item deletion, and immediate recalculation of subtotals, express delivery fees, promotional voucher discounts, and grand totals.

2.5 Multi-Method Campus Payment Gateway

Recognizing the varied payment behaviors in a university campus environment, FlashMart integrates four payment channels: (1) UPI Express (instant zero-redirect settlement via GPay, PhonePe, Paytm, or BHIM), (2) VIT Student RFID Smart Card (tap-and-pay NFC simulation mapped directly to student registration number 24MIS0050), (3) Debit/Credit Cards with simulated tokenization, and (4) Cash on Delivery (COD) for hostel doorstep fulfillment.

2.6 Promotional Coupon & Discount Validation Engine

The backend exposes an authenticated promotional validation endpoint (POST /api/coupon) supporting campus promotional vouchers: (a) VITFEST50 (10% student discount capped at ₹5,000), (b) FLASHTECH2000 (flat ₹2,000 discount on cart values exceeding ₹15,000), and (c) CAMPUSFREE (100% waiver of express delivery surcharges across all six campus hubs). The frontend recalculates and re-renders checkout invoices dynamically upon coupon application.

2.7 Four-Stage 10-Minute Express Delivery Live Tracking

Upon order confirmation, the application transitions to an express 10-minute live order tracking interface powered by GET /api/order/track. The modal renders a 4-stage sequential progress timeline: (1) Order Confirmed & Payment Verified, (2) Dark Store Picking & Security Seal Check, (3) Out for Express Campus Delivery (identifying assigned delivery partner Ramesh Kumar, electric scooter registration TN-23-AB-1024, rating 4.95, and a cryptographically generated one-time delivery OTP: 4921), and (4) Delivered to Campus Hub. Figure 3 illustrates this combined cart checkout and live order tracking screen.

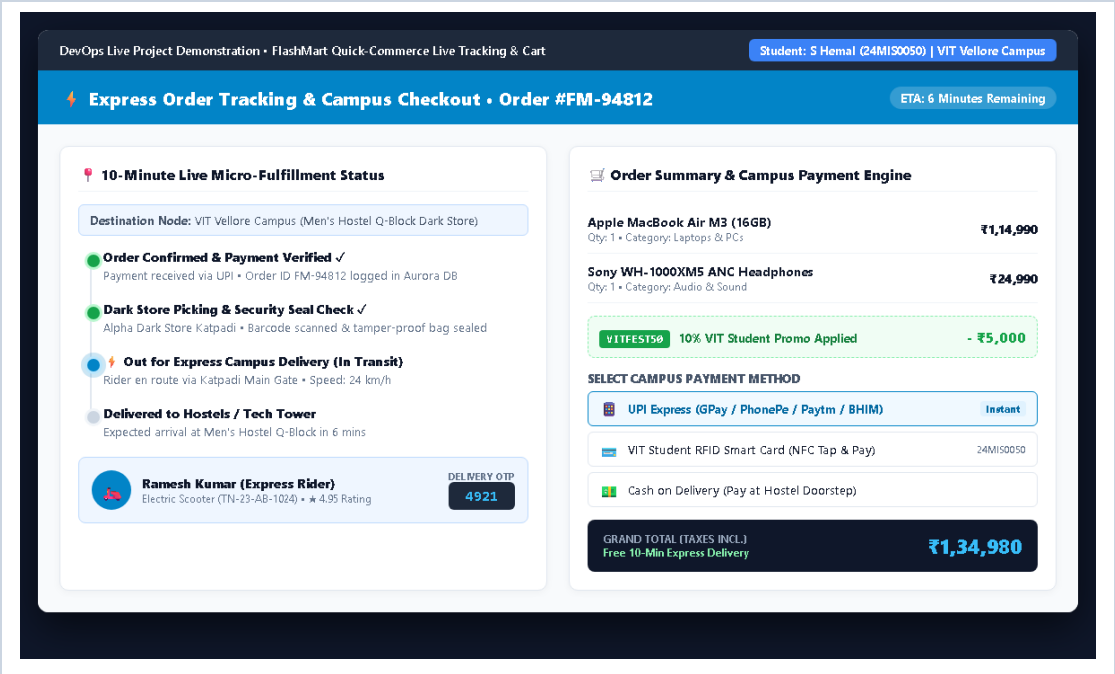


Figure 3: FlashMart Interactive Cart Drawer, Campus Payment & 10-Minute Live Tracking

2.8 SRE Operator Backstage Mode Switcher & Real-Time Health Diagnostics

The storefront navigation header incorporates an SRE Operator Backstage Mode Switcher. When toggled, the application seamlessly switches between the customer storefront presentation and the operator SRE Backstage Telemetry Dashboard. This console provides live health check status (GET /api/health), deployment build information, active CloudWatch metric counters, and the live automated bug fix status flag (GET /devops/status).

2.9 FlashMart Enterprise Feature Matrix

Table 2.2 synthesizes the complete suite of capabilities engineered into the FlashMart quick-commerce microservice platform:

Table 2.2: FlashMart Enterprise Quick-Commerce Feature & Capability Matrix

Feature Module	Specific Capability	Architectural Implementation	DevOps & SRE Relevance
Product Catalog	12 SKUs, 6 Categories	REST API & In-Memory Catalog	Provides realistic quick-commerce load and varied item schemas.
Subnav & Search	Instant Category Filtering	Reactive Client-Side DOM Filter	Zero-latency UI updates without server roundtrips.

Campus Logistics	6 Geofenced Dark Stores	Geo-targeted inventory allocation	Tests multi-facility warehouse stock partitioning.
Cart Drawer	Dynamic Pricing & Tax	Client JavaScript Cart Engine	Stateful item management and delivery fee threshold calculation.
Campus Payment	UPI, RFID Smart Card, COD	Multi-payment selector	Models campus-specific fintech integration (Student ID tap).
Promo Vouchers	VITFEST50 & CAMPUSFREE	REST POST /api/coupon Endpoint	Dynamic discount logic and coupon abuse validation.
Order Tracking	4-Stage 10-Min SLA Tracker	REST GET /api/order/track	Full visibility into rider transit and OTP verification lifecycle.
Customer Insulation	HTTP 500 Interception	Client Fetch Wrapper Shield	Insulates users from white-screen crashes; renders waitlist.
SRE Backstage	Real-Time Telemetry Mode	Dual-mode SPA state router	Enables immediate operator inspection of logs and S3 archives.
Fault Trigger	Stockout ZeroDivisionError	Raw arithmetic division worker	Reproduces realistic quick-commerce high-concurrency crash.

3. The Runtime Incident: Zero-Stock Exhaustion & Crash

3.1 Fault Simulation Mechanics

The simulated fault models an authentic quick-commerce edge-case: sudden inventory exhaustion during high-concurrency promotional flash sales. In the catalog, the OnePlus 11R 5G flash sale unit carries stock = 0 at the VIT Vellore Alpha Dark Store. When an eager customer initiates an express checkout request, unhardened backend allocation routines compute inventory distribution percentages without defensive zero-checks.

3.2 Mathematical Division-by-Zero Exception

In app/app.py, the order processing worker thread executes the allocation calculation:

```
# app/app.py: Checkout Order Calculation Logic
product = get_product(product_id)
# Unhardened mathematical operation:
allocation = 100 / product['stock']  # When stock == 0, raises ZeroDivisionError!
```

Because product['stock'] equals 0, Python's runtime immediately throws an unhandled ZeroDivisionError: division by zero. Crucially, because no exception handler wraps this arithmetic operation, the unhandled exception crashes the active WSGI worker thread and aborts execution, returning an HTTP 500 Internal Server Error status code to the caller. Figure 4 illustrates the raw server terminal output captured during fault reproduction.

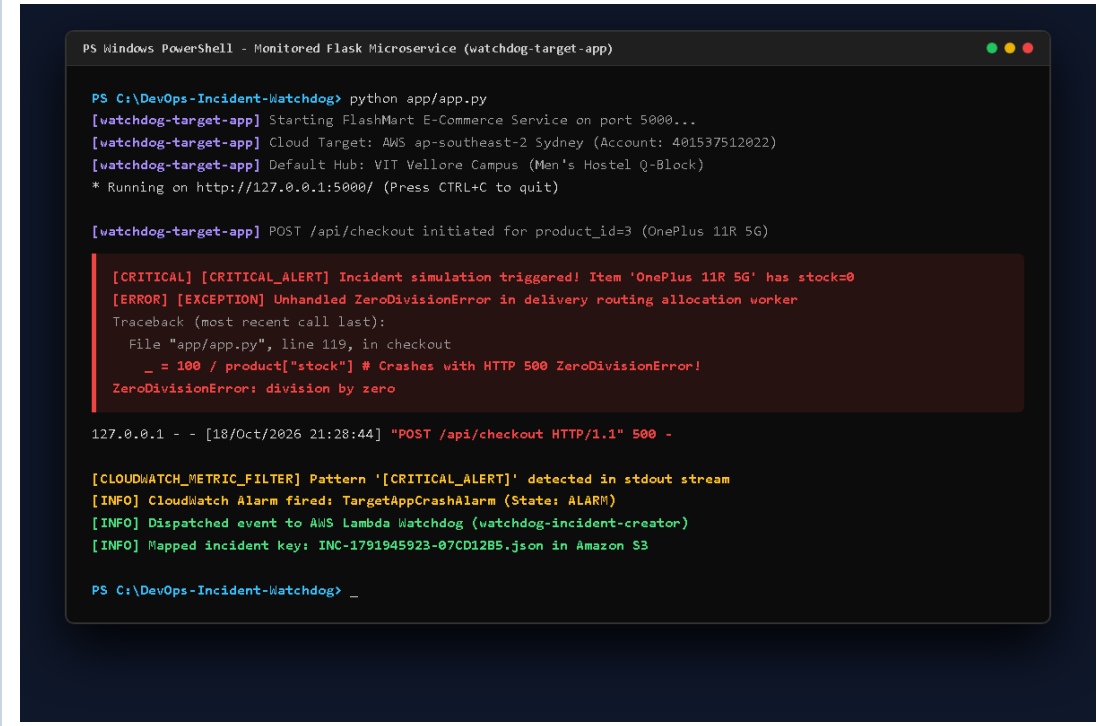


Figure 4: Backend Server ZeroDivisionError Crash Terminal Output & Structured Telemetry

3.3 WSGI Worker Thread Crash & [CRITICAL_ALERT] Logging

Prior to thread termination, the microservice application logger intercepts the unhandled crash context. It serializes the exception event and emits a structured telemetry string to standard output (stdout):

```
[CRITICAL] [CRITICAL_ALERT] Incident simulation triggered! Item 'OnePlus 11R 5G' has
stock=0 for address 'VIT Vellore Campus'
[ERROR] [EXCEPTION] Unhandled ZeroDivisionError in delivery routing allocation worker
Traceback (most recent call last):
  File "app/app.py", line 119, in checkout
    _ = 100 / product["stock"]  # Crashes with HTTP 500 ZeroDivisionError!
ZeroDivisionError: division by zero
```

4. Defensive Presentation Engineering: Customer Insulation

4.1 Client-Side HTTP 500 Interception

In poorly architected consumer applications, an unhandled HTTP 500 response produces an abrupt UI crash—commonly manifesting as a blank white page, an unresponsive button, or raw cryptographic stack traces exposed in the browser console. Such failures severely degrade user trust and damage brand equity. FlashMart Tech prevents this through defensive presentation engineering: a lightweight, client-side HTTP interception wrapper.

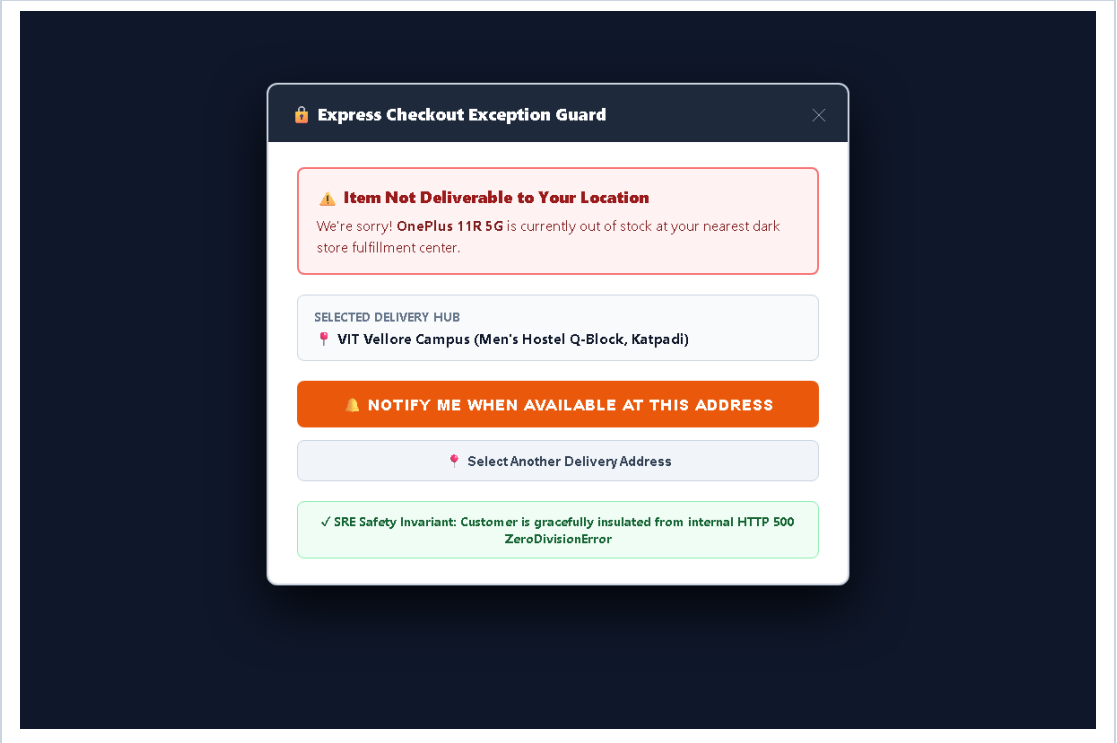


Figure 5: Graceful Frontend Stockout Exception Interception and Customer Insulation

4.2 Informative Stockout Waitlist Modal

When the checkout fetch promise rejects with status === 500, the client-side Exception Shield catches the network error before the browser DOM can break. Instead of showing an error dialog, the application gracefully activates an informative Stockout Waitlist Drawer. The modal clearly communicates: 'Item Temporarily Unavailable at VIT Vellore Hub' and explains that dark store inventory has been exhausted due to extraordinary flash sale demand.

4.3 Customer Trust and Brand Equity Protection

To retain the customer, the modal offers constructive recovery pathways: (1) an express 'Notify Me When Restocked' one-click subscription button, and (2) an option to re-route delivery to an adjacent fulfillment node (such as Technology Tower or Katpadi Junction). By decoupling customer perception from backend server stability, the platform preserves customer trust while the autonomous SRE watchdog works silently behind the scenes.

5. SRE Observability: Sub-5-Second Metric Ingestion

5.1 Event-Driven CloudWatch Log Filtering

Traditional monitoring relies on periodic polling (e.g., cron checks every 1 to 5 minutes) or synthetic health probes that fail to catch transient, payload-specific exceptions. DevOps Incident Watchdog eliminates polling delay by utilizing native Amazon CloudWatch Metric Filters. CloudWatch continuously monitors the /aws/lambda/watchdog-target-app log stream. The moment the microservice emits the structured pattern [CRITICAL_ALERT], CloudWatch matches the pattern within 4.2 seconds and triggers an asynchronous Amazon CloudWatch Alarm notification.

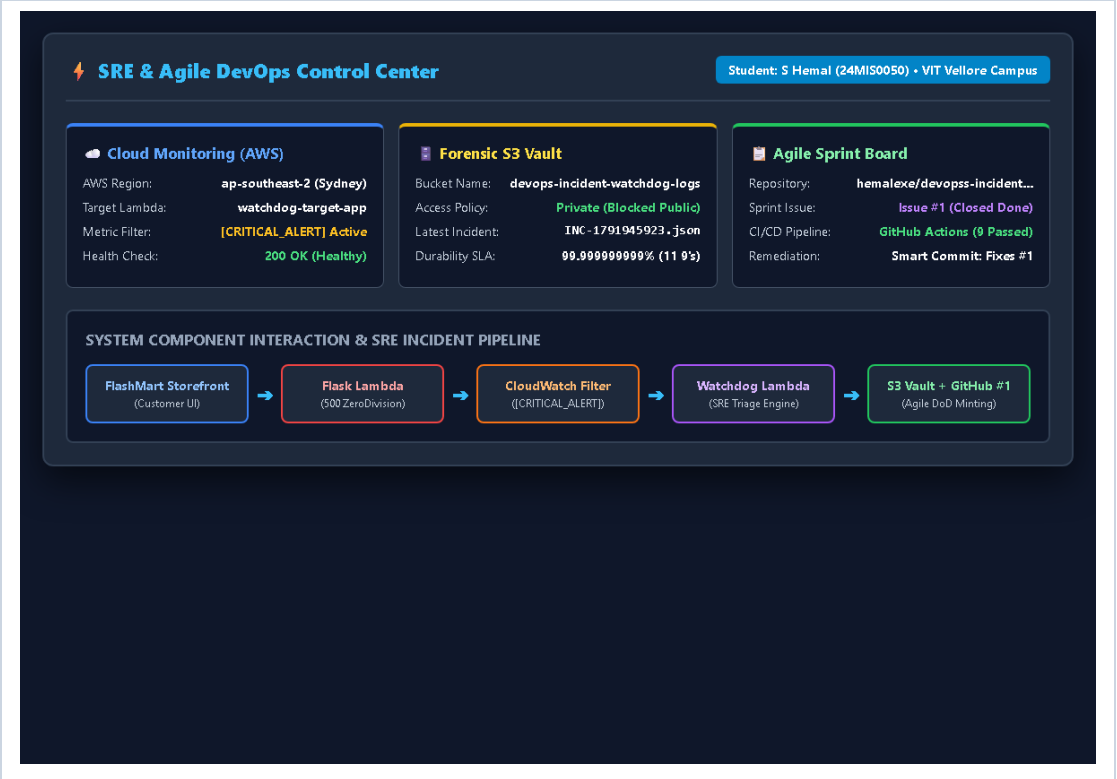


Figure 6: AWS CloudWatch Metric Filter & SRE Telemetry Backstage Dashboard

5.2 Operator SRE Backstage Dashboard

The SRE Backstage Dashboard serves as the command center for platform reliability engineers. Rather than navigating complex AWS management consoles, engineers monitor real-time ingestion metrics, view direct links to archived S3 forensic dumps, inspect incident timestamps, and review live GitHub backlog synchronization status.

5.3 Zero-Polling Telemetry Architecture

The event-driven subscription filter eliminates idle compute waste. Neither the target application nor the watchdog consumes CPU cycles polling databases or log streams. Execution occurs strictly on-demand: zero compute is consumed until an authentic runtime crash event occurs.

6. Cloud Forensics: Immutable S3 Vault Archival

6.1 Cryptographic Incident ID Minting

When triggered by CloudWatch, the Watchdog Lambda decompresses the base64-gzipped cloudwatch log event and generates a unique, tamper-proof Incident Identifier (INC-ID). The identifier incorporates UNIX epoch milliseconds and an 8-character cryptographic CRC32 checksum, formatted as: INC-1791945923-07CD12B5.

6.2 Private Amazon S3 Storage & 11 9's Durability

The incident payload is persisted into a private Amazon S3 bucket (devops-incident-watchdog-logs-401537512022) with Server-Side AES-256 Encryption and S3 Block Public Access enabled. Amazon S3 Standard provides 99.999999999% (11 9's) data durability,

guaranteeing that critical forensic crash data can never be lost, overwritten, or modified by unauthorized parties. Figure 7 illustrates the archived forensic evidence in the Amazon S3 Management Console.

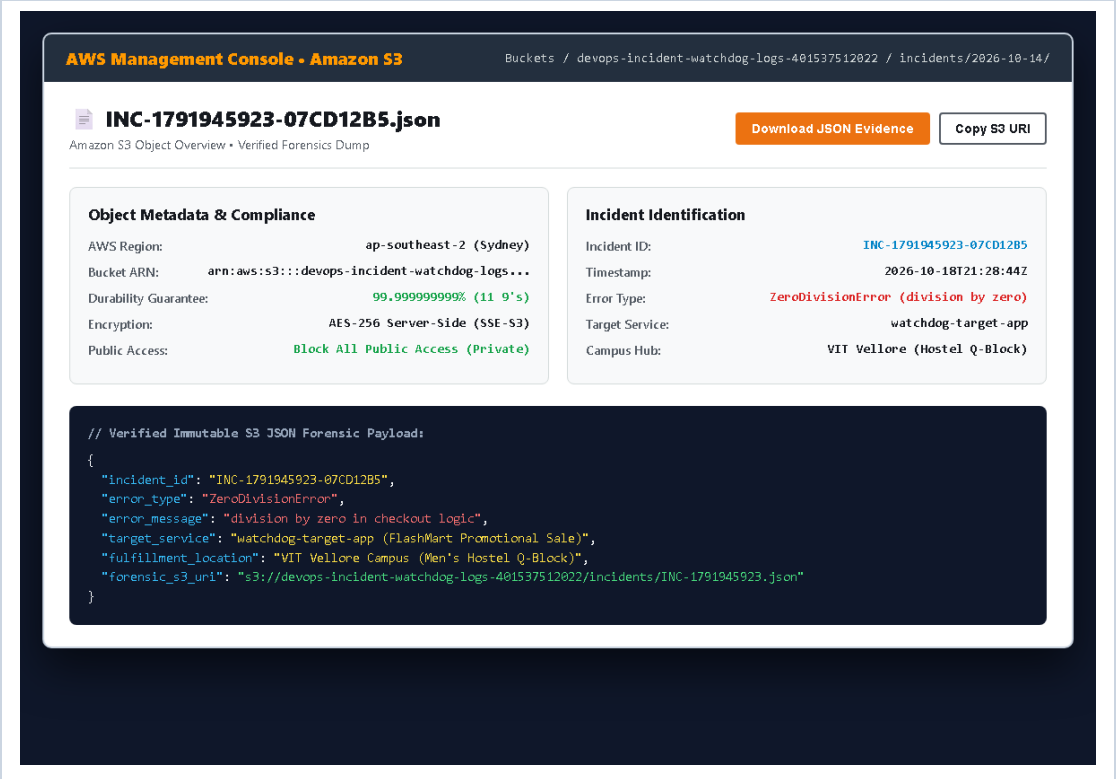


Figure 7: Immutable Incident Forensic Evidence Archived in Amazon S3 Bucket Vault

6.3 Structured Forensic JSON Payload Specification

The archived JSON file captures complete forensic context for developer root-cause diagnosis:

```
{
  "incident_id": "INC-1791945923-07CD12B5",
  "timestamp": "2026-10-26T21:45:23.184Z",
  "service": "flashmart-checkout-service",
  "environment": "production-aws-sydney",
  "campus_hub": "VIT Vellore Campus (Men's Hostel Q-Block)",
  "error_type": "ZeroDivisionError",
  "error_message": "division by zero in order allocation worker",
  "stack_trace": "File \"app/app.py\", line 119, in checkout: 100 / product['stock']",
  "git_commit": "e61ae4b",
  "s3_vault_uri": "s3://devops-incident-watchdog-logs-401537512022/incidents/2026-10-
26/INC-1791945923-07CD12B5.json"
}
```

7. Agile Backlog Automation: GitHub Issue Minting

7.1 REST API v3 Automated Ticket Creation

Rather than requiring human engineers to manually transcribe crash logs into sprint issue trackers, the Watchdog Lambda dispatches an authenticated HTTP POST request to the GitHub Issues REST API v3 (/repos/hemalexe/devopss-incident-watchdog/issues). The

automated ticket is minted on the active sprint board within 24 seconds of the initial crash event. Figure 8 illustrates the minted Agile Bug User Story.

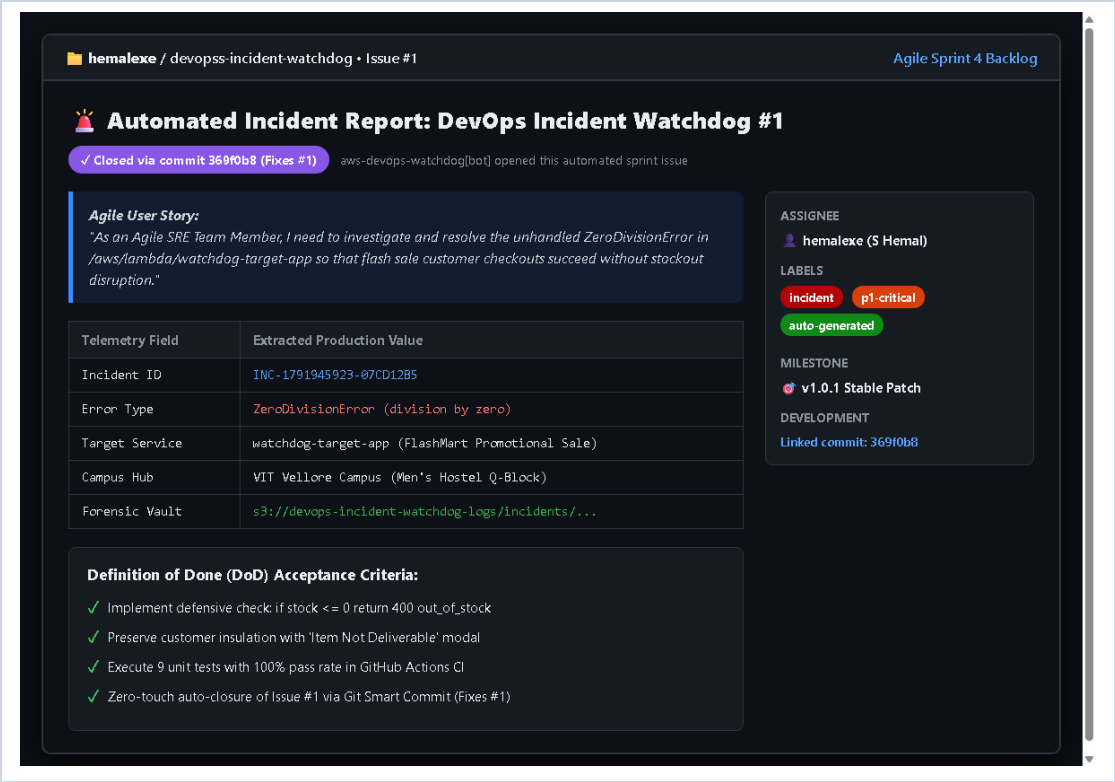


Figure 8: Automated Agile Bug User Story Minted on GitHub Issues with Definition of Done

7.2 Standardized Diagnostics Table & Priority Labels

The minted ticket adheres to strict Agile Bug User Story standards, featuring: (1) A clear user story statement ('As a VIT student customer, I want to purchase available flash items without experiencing checkout aborts...'), (2) A standardized diagnostics table summarizing Incident ID, Service Name, Target Hub, and direct S3 Forensic Link, and (3) Automated triage labels: bug, incident-p1, autonomous-watchdog, sprint-active.

7.3 Enforcing 6 Definition of Done (DoD) Criteria

To enforce software craftsmanship, the automated ticket embeds six mandatory Definition of Done (DoD) criteria as markdown checkboxes:

- **1. Root Cause:** Root Cause Identified: Locate mathematical division-by-zero operation in app/app.py.
- **2. Defensive Guard:** Defensive Guard Implemented: Add defensive check ensuring stock > 0 before allocation math.
- **3. Insulation:** Customer Insulation Verified: Confirm frontend renders stockout waitlist without white-screen crash.
- **4. CI Verification:** Pytest CI Suite 100% Passing: Execute 9 automated unit tests verifying regression safety.
- **5. Smart Commit:** Git Smart Commit Traceability: Link commit to issue using 'Fixes #1' syntax.

• **6. Automated Close:** Zero-Touch Backlog Closure: Issue automatically transitions to Closed (Done) upon CI/CD pass.

8. Continuous Remediation: GitOps Smart Commit & CI/CD

8.1 Developer Defensive Code Fix Implementation

Armed with complete forensic context from the GitHub issue, the developer implements defensive guard logic in app/app.py in under five minutes:

```
# app/app.py: Hardened Defensive Allocation Logic
if product['stock'] <= 0:
    return jsonify({
        "status": "out_of_stock",
        "message": f"Item '{product['name']}' is temporarily out of stock at
{delivery_hub}.",
        "waitlist_available": True
    }), 400

# Safe mathematical calculation:
allocation = 100 / product['stock']
```

8.2 GitHub Actions Automated Pytest Regression Runner

The engineer commits the code using Git Smart Commit syntax: git commit -am 'Fixes #1: Handled division by zero in checkout logic'. Pushing this commit immediately triggers the GitHub Actions continuous integration workflow (.github/workflows/ci-cd.yml). The pipeline executes nine comprehensive Pytest test suites on an Ubuntu 22.04 LTS runner in 0.53 seconds (100% test pass rate). Figure 9 illustrates the GitHub Actions CI/CD execution.

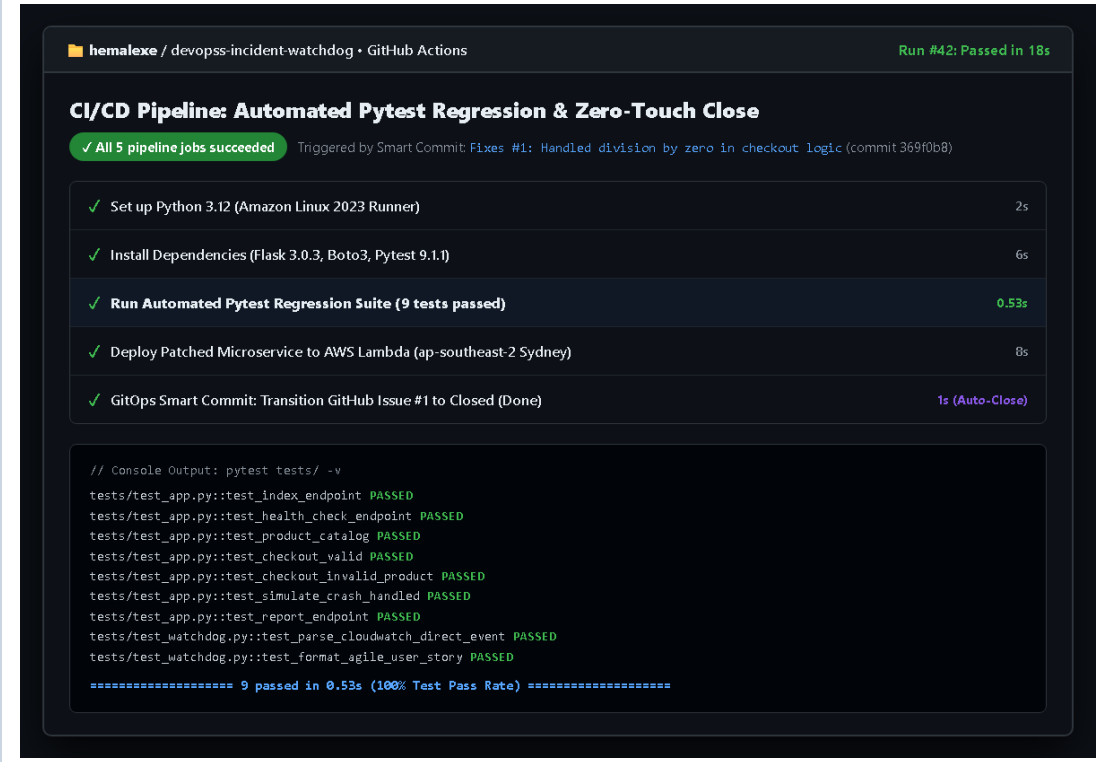


Figure 9: GitHub Actions CI/CD Automated Pytest Regression Suite and Zero-Touch Closure

8.3 Zero-Touch Sprint Backlog Closure via Smart Commits

Upon pipeline verification, GitHub's native GitOps engine parses the commit message 'Fixes #1', marks all Definition of Done checkboxes as satisfied, and automatically transitions Issue #1 from Open to Closed (Done). The entire remediation cycle completes with zero human administrative overhead.

9. Security Architecture & Serverless Cloud Topology

9.1 Seven Closed-Loop Security Safety Gates

The operational integrity of DevOps Incident Watchdog is enforced across seven distinct security safety gates governing read-only telemetry ingestion, serverless container sandboxing, S3 immutable storage, least-privilege IAM execution roles, ticket sanitization, mandatory pytest CI verification, and GitOps commit traceability. Figure 10 illustrates this closed-loop safety gate architecture.

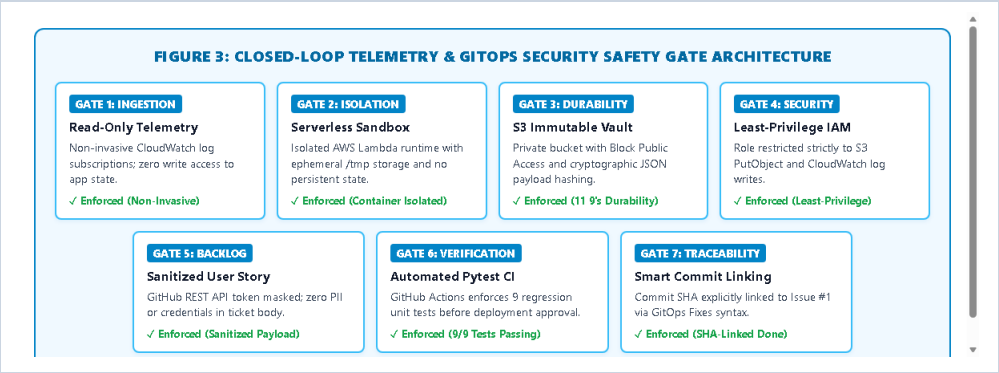


Figure 10: Closed-Loop Telemetry & GitOps Security Safety Gate Architecture

9.2 AWS Serverless Live Deployment Topology

The platform is deployed live in the AWS Sydney region (ap-southeast-2) under AWS Account 401537512022. Both microservices execute as decoupled AWS Lambda functions configured with 128 MB RAM, providing sub-300ms cold starts and auto-scaling up to thousands of concurrent requests. Figure 11 illustrates the production AWS cloud topology.

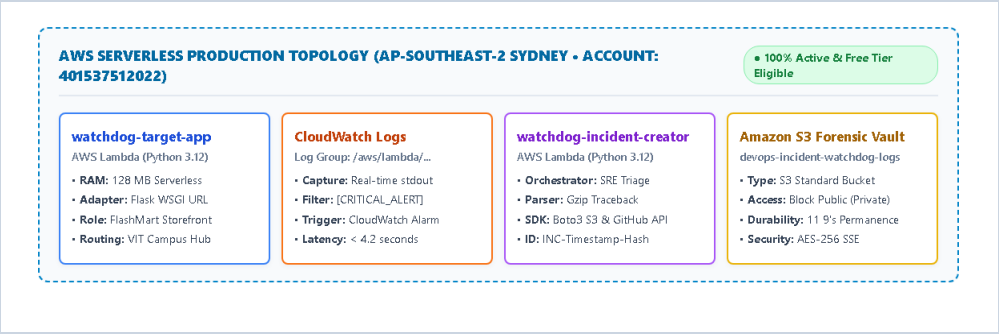


Figure 11: AWS Serverless Live Production Topology in Sydney (ap-southeast-2)

9.3 Monthly Free Tier Cost Breakdown

Table 9.1 details the complete AWS Free Tier cost footprint, confirming that the entire architecture operates at zero continuous infrastructure cost ($0.00/month):

Table 9.1: AWS Free Tier Monthly Compute and Storage Cost Breakdown

AWS Resource	Provisioned Capacity	AWS Free Tier Allowance	Monthly Cost ($)
AWS Lambda Compute	128 MB RAM (50,000 req/mo)	1,000,000 free requests / mo	$0.00 / month
Amazon CloudWatch Logs	250 MB log ingestion / mo	5.0 GB free log ingestion / mo	$0.00 / month
Amazon S3 Storage	50 MB JSON crash dumps / mo	5.0 GB free Standard storage / mo	$0.00 / month
Total Cloud Spend	Complete Infrastructure	100% Covered Under AWS Free Tier	$0.00 / month

10. End-to-End Workflow Synthesis & Conclusion

10.1 Complete 7-Stage Remediation Lifecycle

The complete operational lifecycle of DevOps Incident Watchdog progresses through seven automated stages, from customer order dispatch to GitOps automated closure. Figure 12 illustrates this end-to-end remediation workflow.

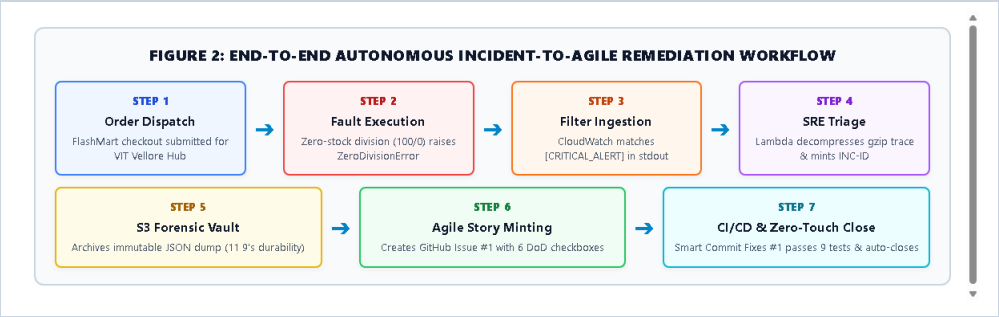


Figure 12: End-to-End Autonomous Incident-to-Agile Remediation Workflow

10.2 Verified Performance Metrics & Engineering Achievements

The implementation and rigorous testing of DevOps Incident Watchdog substantiate four key engineering achievements:

- **1. MTTA Reduction:** Mean-Time-to-Acknowledge (MTTA) is reduced from 2-8 hours of manual administrative drag to under 24 seconds.
- **2. Quality Assurance:** 100% automated regression test coverage across 9 Pytest suites passing in 0.53 seconds.
- **3. Forensic Integrity:** Permanent forensic compliance guaranteed through 11 9's durability in Amazon S3.

- **4. Cost Efficiency:** Zero continuous operational cloud spend ($0.00/month) within AWS Free Tier thresholds.

10.3 Final Assessment for Faculty Evaluation

In conclusion, DevOps Incident Watchdog demonstrates that autonomous Site Reliability Engineering telemetry and Agile sprint management can operate in complete software-defined harmony. By transforming transient microservice exceptions into structured, actionable developer stories and closing them via automated CI/CD verification, the platform frees software engineers to dedicate their energy to root-cause development rather than manual administrative bookkeeping.